

Praktikumsbericht zu Versuch 5

Von Daniel Baer (3235621)

1. Graphviz-Visualisierung

Python Skript: *V05_01.py*

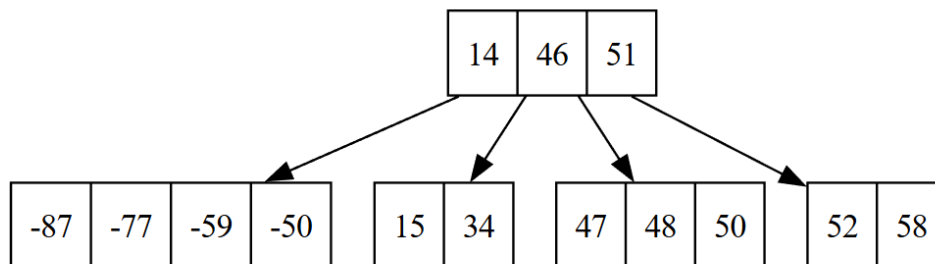


Abbildung 1 Visualisierung des erzeugten B-Baums mit den Werten aus *seq0.txt*

Output:

PDF: [REDACTED] \BTree_O3_seq0.pdf

Root Key Count: 3

Leaf Depths: 1

Keys in Sorted Order: True

- Die Wurzel enthält drei Schlüssel (14, 46, 51)
- Alle Blattknoten liegen auf derselben Ebene (Ebene 1), da bei jedem Hinzufügen eines Elements wird bei einem zu vollen Blatt die Struktur nach oben hin angepasst.
Der Baum wächst nur an der Wurzel und die Blattknoten bleiben auf derselben Ebene.

Zusatz (seq1.txt)

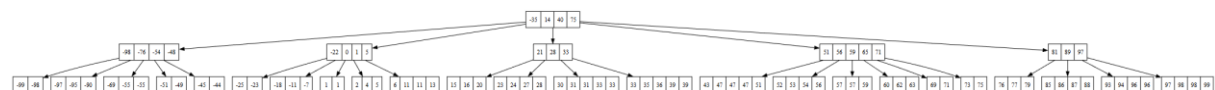


Abbildung 2 Visualisierung des erzeugten B-Baums mit den Werten aus *seq1.txt*

Output:

PDF: [REDACTED] \BTree_O3_seq1.pdf

Root Key Count: 4

Leaf Depths: 2

Keys in Sorted Order: True

- Die Wurzel enthält vier Schlüssel (-35, 14, 40, 75)
- Alle Blattknoten liegen auf derselben Ebene (Ebene 2)

2. Eigenschaften des B-Baums

B-Baum mit Elementen aus seq3.txt

Ordnung 3

Erzeugtes PDF mit Baum-Visualisierung: BTree_O3_seq3.pdf

Output:

```
PDF: [REDACTED] \BTree_O3_seq3.pdf
Root Key Count: 4
Leaf Depths: 6
Keys in Sorted Order: True
Search Result for 0: (0 2 5)
```

- Höhe: 3
- Die Schlüssel werden in sortierter Reihenfolge ausgegeben.
- Inhalt des Knoten mit Schlüssel 0: 0, 2, 5

Ordnung 5

Erzeugtes PDF mit Baum-Visualisierung: BTree_O5_seq3.pdf

Output:

```
PDF: [REDACTED] \BTree_O5_seq3.pdf
Root Key Count: 6
Leaf Depths: 4
Keys in Sorted Order: True
Search Result for 0: (-14 -10 -5 0)
```

- Höhe: 4
- Die Schlüssel werden in sortierter Reihenfolge ausgegeben.
- Inhalt des Knoten mit Schlüssel 0: -14, -10, -5, 0
- Ein Knoten kann bei der Ordnung 5 mehr Schlüssel enthalten als ein Knoten bei Ordnung 3, somit werden bei gleicher Anzahl an Schlüsseln weniger Ebenen benötigt.

3. Disk-I/O und der Parameter m

Python Skript: V05_03.py

Output:

```
Order=2 : height=6 , comparisons=3864 , loads=3223 , saves=1334
Order=3 : height=4 , comparisons=4159 , loads=2226 , saves=944
Order=5 : height=3 , comparisons=5010 , loads=1572 , saves=737
Order=10 : height=2 , comparisons=7045 , loads=1234 , saves=602
Order=20 : height=1 , comparisons=9700 , loads=977 , saves=548
```

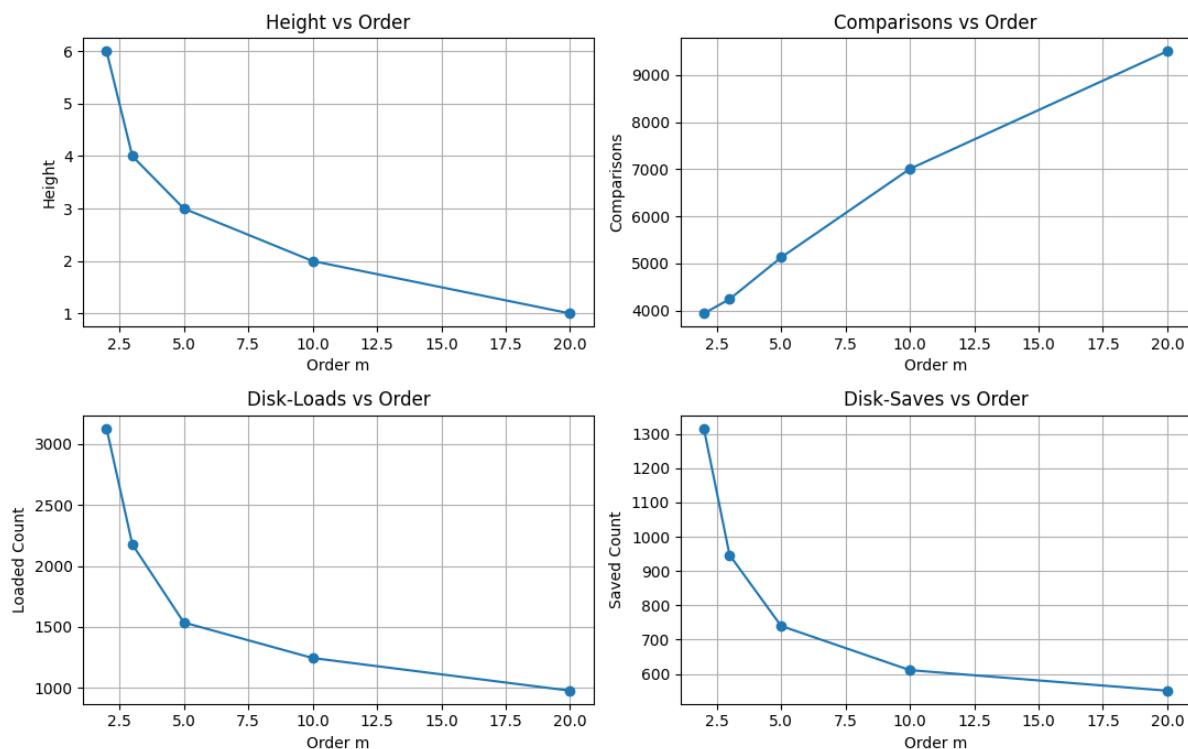


Abbildung 3 Graphen zu Auswertung der Höhe, Vergleiche, Disk-Loads und -Saves in Abhängigkeit der Ordnung

- Mit steigender Ordnung sinkt die Höhe des Baums, so werden weniger Knoten durchlaufen.
- Die Höhe ist $O(\log n)$, die Anzahl der Vergleiche in einem Knoten allerdings $O(m)$. Somit sinkt zwar die Anzahl der Ebenen, es fallen aber mehr Vergleiche pro Ebene an.
- Bei einer Größe für 4kB pro Knoten, lässt sich ein Knoten folgendermaßen aufbauen:
 - $(2m-1)*8$ Byte für Schlüssel
 - $(2m)*8$ Byte für Verweise
- Daraus lässt sich herleiten, dass $(4m-1)*8$ Byte maximal 4kB groß sein darf.
 - $(4m-1)*8 \text{ Byte} \leq 4096 \text{ Byte}$
 - $m \leq 128$
- Die Optimale Ordnung ist somit $m = 128$